Homework 09: SVM Analysis

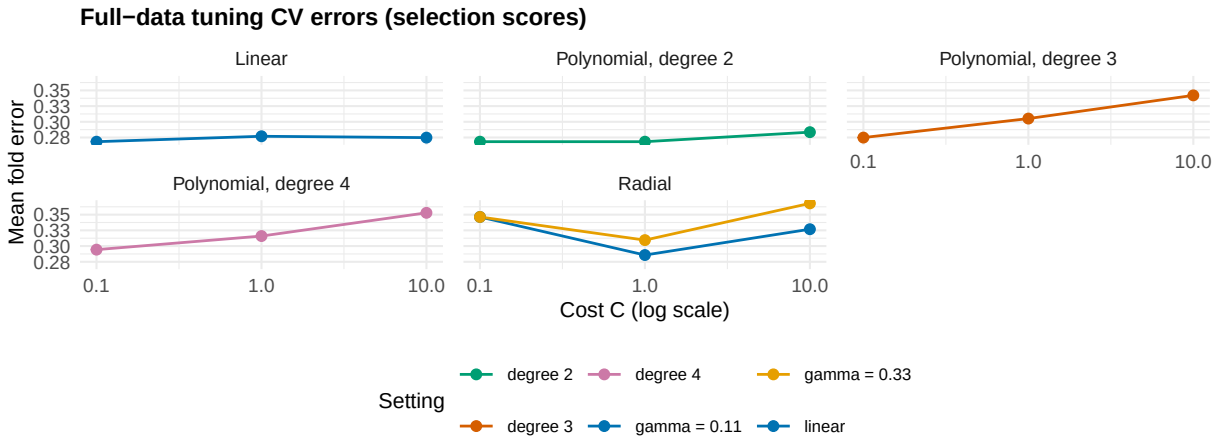
Skill-guided analysis of Question 1

Data and suitability. 462 complete observations; categorical CHD response (0: 302, 1: 160) and nine numeric predictors: sbp, tobacco, ldl, adiposity, famhist, typea, obesity, alcohol, age. Family history is coded Absent = 1, Present = 2. No missing values were found; numeric predictors were centered/scaled within each training fold.

Tuning and selection. Seed 43209; 5 stratified outer folds estimate the nested selection procedure, with 5 stratified inner folds tuning each outer-training sample. The final setting is chosen by five-fold CV on all observations. Compared 18 settings: linear, polynomial degrees 2–4, and radial; $C \in \{0.1, 1, 10\}$. Polynomial $\gamma = 1/9$, coef0= 1; radial $\gamma \in \{1/9, 1/3\}$. No class weights were used. The table/figure show tuning scores, not unbiased performance.

Selected: linear SVM ($C = 0.1$), full-data tuning-CV error 0.2683. Nested CV of the complete selection procedure: error 0.3009; accuracy 0.6991; balanced accuracy 0.6259. Outer-fold class recalls: 0: 261 correct of 302 (recall 0.864); 1: 62 correct of 160 (recall 0.388). The selected setting was chosen in 1/5 outer folds.

Kernel	Degree	C	γ	Tuning CV error	Selected
Linear	–	0.1	–	0.2683	*
Linear	–	1	–	0.2770	
Linear	–	10	–	0.2748	
Polynomial (degree 2)	2	0.1	0.111	0.2684	
Polynomial (degree 2)	2	1	0.111	0.2683	
Polynomial (degree 2)	2	10	0.111	0.2835	
Polynomial (degree 3)	3	0.1	0.111	0.2749	
Polynomial (degree 3)	3	1	0.111	0.3051	
Polynomial (degree 3)	3	10	0.111	0.3421	
Polynomial (degree 4)	4	0.1	0.111	0.2943	
Polynomial (degree 4)	4	1	0.111	0.3159	
Polynomial (degree 4)	4	10	0.111	0.3529	
Radial	–	0.1	0.111	0.3463	
Radial	–	1	0.111	0.2857	
Radial	–	10	0.111	0.3268	
Radial	–	0.1	0.333	0.3463	
Radial	–	1	0.333	0.3095	
Radial	–	10	0.333	0.3680	



Interpretation and limitations. linear at C=0.1 is the minimum tuning-CV setting; degree-2 polynomial at C=1 ties at four decimal places. The setting was selected in 1/5 outer folds, so its identity is not stable across folds. Nested CV estimates selection-procedure performance; the full-data tuning score is optimistic. Positive-class recall is materially lower than negative-class recall. Results depend on this candidate grid and fold assignment; there is no independent test set.

Appendix: SVM model-fitting skill

```
---
name: svm-model-fitting
description: "Use only when explicitly invoked to assess data suitability, fit and tune support vector machine
user-invocable: true
disable-model-invocation: true
---

# Skill for SVM Model Fitting

## Invocation and scope
Use this skill only when explicitly invoked. Require the user to identify the dataset, categorical response, ar

## Analysis procedure
1. Inspect sample size, response classes and counts, predictor types, missingness, and potential encoding or sc
2. Compare a linear kernel, polynomial kernels of degrees 2, 3, and 4, and a radial basis kernel. Explain that
3. Set and report a random seed. Define a manageable candidate grid before fitting: tune cost \(C\) for every k
4. Use stratified cross-validation with the same folds for all candidates. Prefer nested cross-validation: tune
5. Within every resample, fit predictor encoding, imputation, centering, and scaling on that fold's training po
6. Use misclassification rate as the primary selection measure; also report accuracy and, when class imbalance
7. Report failed fits and their impact instead of silently omitting candidates. Select settings using training

## Required report
Prioritize the evidence needed to justify the final model in no more than one printed page at 11-point font. In

## Final checks
Verify all candidates used the same folds; preprocessing was fitted within training folds; tuning did not use o
```

Appendix: Complete R code

```
#!/usr/bin/env Rscript

suppressPackageStartupMessages({
  library(e1071)
  library(ggplot2)
})

script_arg <- grep("^--file=", commandArgs(trailingOnly = FALSE), value = TRUE)
if (length(script_arg) != 1L) {
  stop("Run this file with Rscript so its location can be identified.")
}
script_file <- normalizePath(sub("^--file=", "", script_arg))
output_dir <- dirname(script_file)
data_file <- file.path(output_dir, "..", "week-08", "data", "saheart.csv")
report_file <- file.path(output_dir, "homework-09.pdf")
plot_file <- file.path(output_dir, "cv-rates-plot.pdf")
results_file <- file.path(output_dir, "cv-rates.csv")
predictions_file <- file.path(output_dir, "nested-cv-predictions.csv")
fold_errors_file <- file.path(output_dir, "full-cv-fold-errors.csv")
skill_file <- file.path(output_dir, "..", "..", ".github", "skills",
                        "svm-model-fitting", "SKILL.md")
if (!file.exists(skill_file)) {
  stop("The SVM skill file is missing; cannot append it to the report.")
}

predictor_names <- c(
  "sbp", "tobacco", "ldl", "adiposity", "famhist",
  "typea", "obesity", "alcohol", "age"
)
required_names <- c(predictor_names, "chd")
heart <- read.csv(data_file, stringsAsFactors = FALSE)
if (!all(required_names %in% names(heart))) {
  stop("The dataset is missing one or more required columns.")
}
heart <- heart[required_names]
if (anyNA(heart)) {
  stop("The dataset contains missing values; no missing-data procedure was specified.")
}
if (!all(heart$chd %in% c(0, 1))) {
  stop("The response chd must be coded 0/1.")
}
if (!all(heart$famhist %in% c("Absent", "Present"))) {
  stop("The famhist values must be Absent or Present.")
}
heart$famhist <- ifelse(heart$famhist == "Absent", 1, 2)
heart$chd <- factor(heart$chd, levels = c(0, 1))
if (!all(vapply(heart[predictor_names], is.numeric, logical(1)))) {
  stop("All predictors must be numeric after the specified encoding.")
}

seed <- 43209L
fold_count <- 5L
set.seed(seed)
make_stratified_folds <- function(response, k) {
  if (min(table(response)) < k) {
    stop("Each response class must have at least as many cases as folds.")
  }
  fold_id <- integer(length(response))
```

```
  for (class_label in levels(response)) {
    indices <- sample(which(response == class_label))
    fold_id[indices] <- rep(seq_len(k), length.out = length(indices))
  }
  fold_id
}

cost_values <- c(0.1, 1, 10)
polynomial_gamma <- 1 / length(predictor_names)
polynomial_candidates <- do.call(
  rbind,
  lapply(2:4, function(degree_value) {
    data.frame(
      kernel = "polynomial",
      degree = degree_value,
      cost = cost_values,
      gamma = polynomial_gamma
    )
  })
)
radial_candidates <- do.call(
  rbind,
  lapply(c(1 / 9, 1 / 3), function(gamma_value) {
    data.frame(
      kernel = "radial",
      degree = NA_integer_,
      cost = cost_values,
      gamma = gamma_value
    )
  })
)
candidates <- rbind(
  data.frame(
    kernel = "linear",
    degree = NA_integer_,
    cost = cost_values,
    gamma = NA_real_
  ),
  polynomial_candidates,
  radial_candidates
)
if (nrow(candidates) > 20L) {
  stop("The candidate grid exceeds the 20-setting limit.")
}

fit_predict <- function(training, validation, setting) {
  training_x <- as.matrix(training[predictor_names])
  validation_x <- as.matrix(validation[predictor_names])
  centers <- colMeans(training_x)
  scales <- apply(training_x, 2, sd)
  if (any(!is.finite(scales)) || any(scales == 0)) {
    stop("A predictor has zero or invalid scale in a training fold.")
  }
  training_x <- scale(training_x, center = centers, scale = scales)
  validation_x <- scale(validation_x, center = centers, scale = scales)
  degree_value <- if (is.na(setting$degree)) 3L else setting$degree
  gamma_value <- if (is.na(setting$gamma)) polynomial_gamma else setting$gamma
  coef0_value <- if (setting$kernel == "polynomial") 1 else 0
  model <- e1071::svm(
    x = training_x,
```

```
    y = training$chd,
    type = "C-classification",
    kernel = setting$kernel,
    cost = setting$cost,
    gamma = gamma_value,
    degree = degree_value,
    coef0 = coef0_value,
    scale = FALSE
  )
  predict(model, newdata = validation_x)
}

cross_validate_candidates <- function(data, fold_id) {
  k <- max(fold_id)
  errors <- matrix(NA_real_, nrow = k, ncol = nrow(candidates))
  for (fold in seq_len(k)) {
    analysis <- data[fold_id != fold, , drop = FALSE]
    validation <- data[fold_id == fold, , drop = FALSE]
    for (candidate in seq_len(nrow(candidates))) {
      predicted <- fit_predict(
        analysis, validation, candidates[candidate, , drop = FALSE]
      )
      errors[fold, candidate] <- mean(predicted != validation$chd)
    }
  }
  list(fold_errors = errors, mean_errors = colMeans(errors))
}

# Nested CV estimates the performance of selecting settings by inner CV.
outer_fold_id <- make_stratified_folds(heart$chd, fold_count)
outer_prediction <- factor(rep(NA_character_, nrow(heart)),
                           levels = levels(heart$chd))
outer_selected <- integer(fold_count)
for (outer_fold in seq_len(fold_count)) {
  outer_training <- heart[outer_fold_id != outer_fold, , drop = FALSE]
  outer_validation <- heart[outer_fold_id == outer_fold, , drop = FALSE]
  inner_fold_id <- make_stratified_folds(outer_training$chd, fold_count)
  inner_cv <- cross_validate_candidates(outer_training, inner_fold_id)
  selected_inner <- which.min(inner_cv$mean_errors)
  outer_selected[outer_fold] <- selected_inner
  outer_prediction[outer_fold_id == outer_fold] <- fit_predict(
    outer_training, outer_validation,
    candidates[selected_inner, , drop = FALSE]
  )
}

# The full-data CV grid selects the final setting; nested CV remains the
# performance estimate for the complete setting-selection procedure.
full_fold_id <- make_stratified_folds(heart$chd, fold_count)
full_cv <- cross_validate_candidates(heart, full_fold_id)
selected_index <- which.min(full_cv$mean_errors)
selected <- candidates[selected_index, , drop = FALSE]
outer_confusion <- table(Actual = heart$chd, Predicted = outer_prediction)
outer_error <- mean(outer_prediction != heart$chd)
outer_accuracy <- mean(outer_prediction == heart$chd)
class_recall <- diag(outer_confusion) / rowSums(outer_confusion)
outer_balanced_accuracy <- mean(class_recall)
write.csv(
  data.frame(
    candidate = rep(seq_len(nrow(candidates)), each = fold_count),
```

```
    fold = rep(seq_len(fold_count), times = nrow(candidates)),
    misclassification_rate = as.vector(full_cv$fold_errors)
  ),
  fold_errors_file,
  row.names = FALSE
)
write.csv(
  data.frame(
    observation = seq_len(nrow(heart)),
    outer_fold = outer_fold_id,
    actual = heart$chd,
    predicted = outer_prediction
  ),
  predictions_file,
  row.names = FALSE
)

candidate_labels <- ifelse(
  candidates$kernel == "polynomial",
  paste0("Polynomial (degree ", candidates$degree, ")"),
  ifelse(candidates$kernel == "radial", "Radial", "Linear")
)
results <- data.frame(
  Kernel = candidate_labels,
  Degree = ifelse(is.na(candidates$degree), "--", candidates$degree),
  Cost = candidates$cost,
  Gamma = ifelse(is.na(candidates$gamma), "--", sprintf("%.3f", candidates$gamma)),
  `Mean tuning CV error` = sprintf("%.4f", full_cv$mean_errors),
  Selected = ifelse(seq_len(nrow(candidates)) == selected_index, "*", ""),
  check.names = FALSE
)
write.csv(results, results_file, row.names = FALSE)

plot_data <- data.frame(
  kernel = candidates$kernel,
  degree = candidates$degree,
  cost = candidates$cost,
  gamma = candidates$gamma,
  cv_error = full_cv$mean_errors
)
plot_data$panel <- ifelse(
  plot_data$kernel == "polynomial",
  paste0("Polynomial, degree ", plot_data$degree),
  ifelse(plot_data$kernel == "radial", "Radial", "Linear")
)
plot_data$curve <- ifelse(
  plot_data$kernel == "polynomial",
  paste0("degree ", plot_data$degree),
  ifelse(
    plot_data$kernel == "radial",
    paste0("gamma = ", format(plot_data$gamma, digits = 2)),
    "linear"
  )
)
plot_data$panel <- factor(
  plot_data$panel,
  levels = c(
    "Linear", "Polynomial, degree 2", "Polynomial, degree 3",
    "Polynomial, degree 4", "Radial"
  )
```

```
)
cv_plot <- ggplot(
  plot_data,
  aes(x = cost, y = cv_error, color = curve, group = curve)
) +
  geom_line(linewidth = 0.55) +
  geom_point(size = 1.8) +
  facet_wrap(~panel, ncol = 3) +
  scale_x_log10(breaks = cost_values) +
  scale_y_continuous(labels = function(x) sprintf("%.2f", x)) +
  scale_color_manual(values = c(
    "linear" = "#0072B2",
    "degree 2" = "#009E73",
    "degree 3" = "#D55E00",
    "degree 4" = "#CC79A7",
    "gamma = 0.11" = "#0072B2",
    "gamma = 0.33" = "#E69F00"
  )) +
  labs(
    title = "Full-data tuning CV errors (selection scores)",
    x = "Cost C (log scale)",
    y = "Mean fold error",
    color = "Setting"
  ) +
  theme_minimal(base_size = 9) +
  theme(
    legend.position = "bottom",
    strip.text = element_text(size = 8),
    plot.title = element_text(size = 10, face = "bold"),
    axis.title = element_text(size = 9),
    axis.text = element_text(size = 8),
    panel.spacing = grid::unit(0.6, "lines")
  )
ggsave(plot_file, cv_plot, width = 7.15, height = 2.75, units = "in")

class_counts <- table(heart$chd)
selected_kernel <- if (selected$kernel == "polynomial") {
  paste0("polynomial, degree ", selected$degree)
} else {
  selected$kernel
}
selected_gamma_text <- if (selected$kernel == "radial") {
  sprintf(", gamma = %.3f", selected$gamma)
} else if (selected$kernel == "polynomial") {
  ", gamma = 1/9, coef0 = 1"
} else {
  ""
}
selected_frequency <- sum(outer_selected == selected_index)
runner_up_index <- order(full_cv$mean_errors)[2]
runner_up <- candidates[runner_up_index, , drop = FALSE]
runner_up_label <- if (runner_up$kernel == "polynomial") {
  paste0("degree-", runner_up$degree, " polynomial")
} else {
  runner_up$kernel
}
score_gap <- full_cv$mean_errors[runner_up_index] -
  full_cv$mean_errors[selected_index]
gap_text <- if (round(score_gap, 4) == 0) {
  "ties at four decimal places"
```

```
} else {
  sprintf("is within %.4f error", score_gap)
}
interpretation <- sprintf(
  paste0(
    "%s at C=%s is the minimum tuning-CV setting; %s at C=%s %s. ",
    "The setting was selected in %d/%d outer folds, so its identity ",
    "is not stable across folds."
  ),
  selected_kernel, format(selected$cost, trim = TRUE),
  runner_up_label, format(runner_up$cost, trim = TRUE), gap_text,
  selected_frequency, fold_count
)
table_rows <- vapply(seq_len(nrow(results)), function(i) {
  sprintf(
    "%s & %s & %s & %s & %s & %s \\\\",
    results$Kernel[i],
    results$Degree[i],
    results$Cost[i],
    results$Gamma[i],
    results$`Mean tuning CV error`[i],
    results$Selected[i]
  )
}, character(1))
confusion_rows <- paste(
  sprintf(
    "%s: %d correct of %d (recall %.3f)",
    rownames(outer_confusion),
    diag(outer_confusion),
    rowSums(outer_confusion),
    class_recall
  ),
  collapse = "; "
)
latex_lines <- c(
  "\\documentclass[11pt]{article}",
  "\\usepackage[margin=0.48in]{geometry}",
  "\\usepackage{graphicx}",
  "\\usepackage{booktabs}",
  "\\usepackage{array}",
  "\\usepackage{xcolor}",
  "\\usepackage{verbatim}",
  "\\pagecolor{white}",
  "\\setlength{\\parindent}{0pt}",
  "\\setlength{\\parskip}{3pt}",
  "\\pagestyle{empty}",
  "\\begin{document}",
  "\\begin{center}",
  "{\\Large\\bfseries Homework 09: SVM Analysis}\\\\[-1pt]",
  "{\\small Skill-guided analysis of Question 1}",
  "\\end{center}",
  sprintf(
    paste0(
      "\\textbf{Data and suitability.}\\ %d complete observations; ",
      "categorical CHD response (0: %d, 1: %d) and nine numeric predictors: ",
      "%s. Family history is coded Absent = 1, Present = 2. No missing ",
      "values were found; numeric predictors were centered/scaled within ",
      "each training fold.\\par"
    ),
    nrow(heart), class_counts["0"], class_counts["1"],
```

```
  paste(predictor_names, collapse = ", ")
),
sprintf(
  paste0(
    "\\textbf{Tuning and selection.}\\ Seed %d; %d stratified outer folds ",
    "estimate the nested selection procedure, with %d stratified inner ",
    "folds tuning each outer-training sample. The final setting is chosen ",
    "by five-fold CV on all observations. Compared %d settings: linear, ",
    "polynomial degrees 2--4, and radial; $C \\in \\{0.1,1,10\\}$. ",
    "Polynomial $\\gamma=1/9$, coef0$=1$; radial $\\gamma \\in \\{1/9,1/3\\}$. ",
    "No class weights were used. The table/figure show tuning scores, not ",
    "unbiased performance.\\par"
  ),
  seed, fold_count, fold_count, nrow(candidates)
),
sprintf(
  paste0(
    "\\textbf{Selected: %s SVM}\\ ($C=%s%s$), full-data tuning-CV error ",
    "%.4f. Nested CV of the complete selection procedure: error %.4f; ",
    "accuracy %.4f; balanced accuracy %.4f. Outer-fold class recalls: %s. ",
    "The selected setting was chosen in %d/%d outer folds.\\par"
  ),
  selected_kernel, format(selected$cost, trim = TRUE), selected_gamma_text,
  full_cv$mean_errors[selected_index], outer_error, outer_accuracy,
  outer_balanced_accuracy, confusion_rows, selected_frequency, fold_count
),
"\\vspace{-2pt}",
"\\begin{center}",
"\\footnotesize",
"\\begin{tabular}{@{}lrrrlc@{}}",
"\\toprule",
"Kernel & Degree & $C$ & $\\gamma$ & Tuning CV error & Selected \\\\",
"\\midrule",
table_rows,
"\\bottomrule",
"\\end{tabular}",
"\\end{center}",
"\\vspace{-3pt}",
"\\begin{center}",
sprintf(
  "\\includegraphics[width=\\textwidth,height=2.45in,keepaspectratio]{%s}",
  basename(plot_file)
),
"\\end{center}",
"\\vspace{-2pt}",
paste0(
  "\\textbf{Interpretation and limitations.}\\ ", interpretation,
  " Nested CV estimates selection-procedure performance; the full-data ",
  "tuning score is optimistic. Positive-class recall is materially lower ",
  "than negative-class recall. Results depend on this candidate grid and ",
  "fold assignment; there is no independent test set."
),
"\\newpage",
"\\section*{Appendix: SVM model-fitting skill}",
"\\small",
"\\verbatiminput{../../.github/skills/svm-model-fitting/SKILL.md}",
"\\newpage",
"\\section*{Appendix: Complete R code}",
"\\small",
"\\verbatiminput{homework-09.R}",
```

```
  "\\end{document}"
)

tex_file <- file.path(output_dir, "homework-09.tex")
writeLines(latex_lines, tex_file)
old_wd <- getwd()
on.exit(setwd(old_wd), add = TRUE)
setwd(output_dir)
pdf_status <- system2(
  "pdflatex",
  args = c("-interaction=nonstopmode", "-halt-on-error", basename(tex_file)),
  stdout = TRUE,
  stderr = TRUE
)
if (!is.null(attr(pdf_status, "status")) && attr(pdf_status, "status") != 0) {
  stop(paste(pdf_status, collapse = "\n"))
}
if (!file.exists(report_file)) {
  stop("LaTeX completed without creating the expected PDF.")
}
unlink(c(
  tex_file,
  file.path(output_dir, "homework-09.aux"),
  file.path(output_dir, "homework-09.log")
))

cat(
  sprintf("Data: %d observations; CHD=0: %d, CHD=1: %d\n",
          nrow(heart), class_counts["0"], class_counts["1"]),
  sprintf("Candidates: %d\n", nrow(candidates)),
  sprintf("Selected: %s, C=%s%s, tuning CV error=%.4f\n",
          selected_kernel, format(selected$cost, trim = TRUE),
          selected_gamma_text, full_cv$mean_errors[selected_index]),
  sprintf("Nested CV: error=%.4f, accuracy=%.4f, balanced accuracy=%.4f\n",
          outer_error, outer_accuracy, outer_balanced_accuracy),
  sprintf("PDF: %s\n", report_file),
  sep = ""
)
```